

Worksheet Validación URDF — Hito 1, Semana 6

IMT-342 | Grupo VEVAS

A. Identificación

Grupo: VEVAS

Integrantes: _____

URDF File / Versión: `urdf/irb120.urdf` , tag `v1.0-hito1` , commit `e077c6c`

Base Frame: `base_link`

Terminal / Flange Frame: `tool0`

Repositorio: `github.com/FlorenciaFrigerio/pfi-hito1-vevas`

Trabajamos con dos archivos URDF. Uno lo escribimos a mano, con comentarios que indican de dónde sale cada número. El otro lo genera el script `generate_urdf.py` a partir de la tabla D-H. Los dos describen el mismo robot y hay un script que lo comprueba.

B. Validez XML / URDF

Herramienta / Método: `check_urdf urdf/irb120.urdf (urdfdom)`

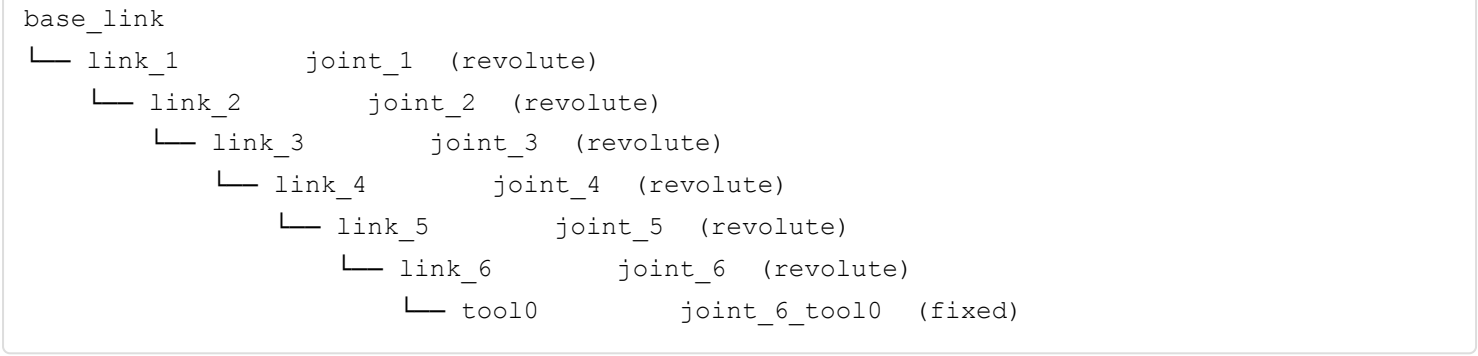
Resultado: **Pass**

Error y Corrección:

La versión entregada no da errores. Durante el desarrollo sí tuvimos un problema que costó encontrar: si el `CMakeLists.txt` no instala los directorios `urdf/` , `meshes/` y `launch/` dentro de `share/` , el `check_urdf` igual pasa y el árbol TF se arma bien, pero las rutas `package://` no resuelven y RViz no muestra ninguna geometría. No da ningún mensaje de error. Lo arreglamos agregando el bloque `install(DIRECTORY ...)` .

Como validación adicional corrimos `compare_urdf.py` , que compara los `<origin>` de las dos versiones del URDF. Las 7 juntas dan diferencia `0,00e+00 m` y `0,00e+00 rad`.

C. Kinematic Tree



Checklist:

- [x] Conexiones completas
- [x] Cero ramas accidentales
- [x] Nombres exactos
- [x] Flange identificable

La salida de `check_urdf` dice `root Link: base_link has 1 child(ren)` y baja hasta `tool0`. Que cada nivel tenga un solo hijo confirma que no hay ramas sueltas. `tool0` está unido por una junta `fixed`, no `revolute`.

D. Evidencia de Definición de Juntas

Los `origin` no los escribimos a ojo. Salen de aplicar $Origin_i = B_{i-1} \cdot R_z(off_i)$, donde $B_i = T_z(d_i) \cdot T_x(a_i) \cdot R_x(\alpha_i)$ y $B_0 = I$.

Joint	Parent	Child	xyz	rpz	axis	Límite [rad]	Fuente	Justificación física
J1	base_link	link_1	$\begin{smallmatrix} 0 & 0 \\ 0 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 & 0 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 \\ 1 \end{smallmatrix}$	$\begin{smallmatrix} -2.879793 \\ / 2.879793 \end{smallmatrix}$	$B_0 = I$	El marco 1 arranca en el origen de la base. $\pm 165^\circ$ del manual. d_1 es la altura de la columna base.
J2	link_1	link_2	$\begin{smallmatrix} 0 & 0 \\ 0.290 \end{smallmatrix}$	$\begin{smallmatrix} -1.5708 \\ -1.5708 \\ 0 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 \\ 1 \end{smallmatrix}$	$\begin{smallmatrix} -1.919862 \\ / 1.919862 \end{smallmatrix}$	$B_1 \cdot R_z(-\pi/2)$	El offset $-\pi/2$ acomoda la diferencia entre el cero de ABB y el de D-H. $\pm 110^\circ$.
J3	link_2	link_3	$\begin{smallmatrix} 0.270 \\ 0 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 & 0 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 \\ 1 \end{smallmatrix}$	$\begin{smallmatrix} -1.919862 \\ / 1.221730 \end{smallmatrix}$	B_2	a_2 es el largo del brazo. $\alpha_2 = 0$ porque z_1 y z_2 son paralelos. Única junta asimétrica: $-110^\circ/+70^\circ$.
J4	link_3	link_4	$\begin{smallmatrix} 0.070 \\ 0 \end{smallmatrix}$	$\begin{smallmatrix} -1.5708 \\ 0 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 \\ 1 \end{smallmatrix}$	$\begin{smallmatrix} -2.792527 \\ / 2.792527 \end{smallmatrix}$	B_3	a_3 es el offset corto del codo. $\alpha_3 = -\pi/2$ lleva el marco al eje del antebrazo. $\pm 160^\circ$.
J5	link_4	link_5	$\begin{smallmatrix} 0 & 0 \\ 0.302 \end{smallmatrix}$	$\begin{smallmatrix} 1.5708 \\ 0 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 \\ 1 \end{smallmatrix}$	$\begin{smallmatrix} -2.094395 \\ / 2.094395 \end{smallmatrix}$	B_4	d_4 estira el antebrazo hasta el centro de muñeca. Es el único α positivo de la tabla. $\pm 120^\circ$.
J6	link_5	link_6	$\begin{smallmatrix} 0 & 0 \\ 0 \end{smallmatrix}$	$\begin{smallmatrix} -1.5708 \\ 0 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 \\ 1 \end{smallmatrix}$	$\begin{smallmatrix} -6.981317 \\ / 6.981317 \end{smallmatrix}$	B_5	No traslada nada: $a_5 = d_5 = 0$, porque la muñeca es esférica. $\pm 400^\circ$.
—	link_6	tool0	$\begin{smallmatrix} 0 & 0 \\ 0.072 \end{smallmatrix}$	$\begin{smallmatrix} 0 & 0 & 0 \end{smallmatrix}$	fixed	—	B_6	d_6 es el saliente de la brida. Va fija porque después de q_6 ya no hay variables.

Notas sobre la tabla:

Todos los `axis` valen `0 0 1` porque D-H define la variable de junta como rotación alrededor de z_{i-1} . Poner otro eje sería mezclar el marco del CAD con el de D-H.

El rpy de J2 fue el más difícil de obtener. Hay que convertir $R_x(-\pi/2) \cdot R_z(-\pi/2)$ a roll-pitch-yaw, y esa matriz tiene $R[2][0] = 1$, con lo cual `pitch` = $-\pi/2$. Ahí la representación rpy pierde un grado de libertad y roll y yaw dejan de ser independientes. Lo resolvimos con la rama degenerada de la fórmula: fijamos `yaw` = 0 y pasamos todo el giro a `roll`.

Los límites van en radianes. El manual los da en grados, y los 110° de J2 son 1,9199 rad. Si se escribe 110 el robot queda con 17 vueltas de recorrido.

E. Zero / Reference Configuration

$$q^{(0)} = [0, 0, 0, 0, 0, 0]^T$$

Pose esperada cualitativamente: la columna base vertical, el brazo y el antebrazo estirados horizontalmente hacia +x, y la brida al frente con su eje z apuntando en la dirección x de la base.

Pose numérica: `p` = [0,374000, 0,000000, 0,630000] m

$$\begin{aligned}x &= d_4 + d_6 &&= 0,302 + 0,072 &&= 0,374 \text{ m} \\z &= d_1 + a_2 + a_3 &&= 0,290 + 0,270 + 0,070 &&= 0,630 \text{ m}\end{aligned}$$

$$\begin{aligned}R &= \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & 0 \end{bmatrix} && \det(R) = +1\end{aligned}$$

¿Pose cero suficiente para probar corrección? ¿Por qué?

No alcanza.

En `q` = 0 el robot queda en una posición donde los eslabones se alinean con sólo dos ejes del marco base, así que la pose se puede obtener sumando cotas sin que aparezcan componentes cruzadas. Eso hace que un error de signo en algún α , o dos errores que se cancelen entre sí, no se noten.

Tampoco prueba bien los offsets. Con `q`₂ = 0 el offset está presente, pero si estuviera mal compuesto (sumado en vez de pre-multiplicado, por ejemplo) eso recién aparece cuando `q`₂ es distinto de cero.

Y los términos cruzados de las matrices de rotación son cero o uno en esta configuración, así que los errores que dependen de productos de senos y cosenos quedan ocultos.

Lo que sí prueba es que la tabla D-H está bien transcrita, porque la pose predicha sumando cotas del manual coincide con el producto matricial (error 2,08e−18 m). Por eso la usamos como primer control, antes de escribir cualquier otra prueba.

La prueba que sí consideramos suficiente es el barrido aleatorio de la sección G & H: 501 configuraciones validadas en posición y orientación.

F. Pruebas de Juntas Individuales

Movimos cada junta 0,5 rad dejando las otras cinco en cero.

Joint	Demás fijas en	Test Val	Mov. Esperado	Mov. Observado	Match	Notas
J1	q = 0	0,5 rad	Gira todo el robot alrededor del eje vertical. Debe conservar z y el radio.	[0,32822, 0,17931, 0,63000]	[x]	z se mantiene en 0,630 y $\ (x,y)\ = 0,374$.
J2	q = 0	0,5 rad	Baja el brazo: x aumenta, z disminuye.	[0,49122, 0,00000, 0,40907]	[x]	y = 0, el movimiento queda en el plano.
J3	q = 0	0,5 rad	Flexiona el codo, con menos amplitud que J2.	[0,36178, 0,00000, 0,44213]	[x]	Desplazamiento menor que J2, coherente con el brazo de palanca más corto.
J4	q = 0	0,5 rad	No debería mover la posición, sólo la orientación.	[0,37400, 0,00000, 0,63000]	[x]	Igual que en q = 0.
J5	q = 0	0,5 rad	Cabecea la muñeca, mueve poco la brida.	[0,36519, 0,00000, 0,59548]	[x]	Se desplaza 0,0355 m, del orden de d ₆ .
J6	q = 0	0,5 rad	No debería mover la posición.	[0,37400, 0,00000, 0,63000]	[x]	Igual que en q = 0.

Notas:

Que J4 y J6 no muevan `tool0` puede parecer un error pero no lo es. En los dos casos el eje de rotación es colineal con el vector que va desde el origen de esa junta hasta la brida, y rotar alrededor de un eje no desplaza los puntos que están sobre ese eje. Nos sirvió como verificación: si J6 moviera la brida, habría algo mal en α_5 o en d₆.

J5 sí la mueve, pero poco, porque su origen coincide con el centro de muñeca (O4 = O5) y sólo arrastra el saliente d₆ = 0,072 m.

J1 conserva z y el radio, que es lo que corresponde a una rotación pura alrededor del eje vertical.

Los valores se obtienen con `python3 scripts/fk_irb120.py`.

G & H. Configuraciones de Prueba 1 y 2 (Comparación FK)

Test A

$$q^{(A)} = [0,3000, -0,5000, 0,8000, 0,2000, -0,6000, 1,0000]^T \text{ rad}$$

Referencia matemática (`forward_kinematics` , NumPy float64):

```
p(A) = [0,239595, 0,065661, 0,525077] m
```

```
R(A) = [ 0,067650  0,326882  0,942641 ]  
      [ -0,935061 -0,308747  0,174171 ]  
      [ 0,347971 -0,893210  0,284768 ]
```

Evidencia ROS/TF: se publica $q^{(A)}$ en `/joint_states` y se lee con `ros2 run tf2_ros tf2_echo base_link tool0`.

Test B

$q^{(B)} = [-0,7854, 0,5236, -0,3491, 1,0472, 0,7854, -1,5708]^T$ rad (es decir $-45^\circ, +30^\circ, -20^\circ, +60^\circ, +45^\circ, -90^\circ$)

```
p(B) = [0,377861, -0,315508, 0,506423] m
```

```
R(B) = [ 0,459870  0,102799  0,882016 ]  
      [ 0,247237 -0,968823 -0,015990 ]  
      [ 0,852874  0,225420 -0,470948 ]
```

Comparación: ☒ Coherente ☐ Discrepancia ☐ Pendiente relación de marcos

No queda ninguna relación de marcos pendiente. `tool0` es el mismo marco que produce la cadena D-H: la junta fija final aplica $B_6 = T_z(0,072)$, que es la última fila de la tabla. No hay ninguna transformación extra entre el marco 6 y el `tool0` del URDF.

Validación estadística

Dos configuraciones sueltas no cubren el espacio articular, así que armamos un barrido automático (`scripts/validate_tf.py`) que publica configuraciones aleatorias dentro de los límites y compara cada una contra la función analítica.

Métrica	Valor	Umbral
Error posicional máximo	2,459e-10 m	1e-06 m
Error posicional medio	1,498e-10 m	—
Error angular máximo, $\ R_{ana}^T \cdot R_{tf} - I\ _F$	1,092e-09	—
Muestras válidas	501 de 1000	—
Margen	4,1e+03 veces por debajo	—

Para la orientación usamos que $R_{ana}^T \cdot R_{tf}$ da la identidad sólo si las dos rotaciones son iguales, porque en $SO(3)$ la inversa es la transpuesta. Así validamos los 6 grados de libertad de la pose y no sólo los 3 de posición.

Sobre por qué el error es 10^{-10} y no 10^{-16} : la verificación interna del generador, que no pasa por ROS, da $3,68e-16$ m, que es el épsilon de float64. La medición vía TF da 10^{-10} porque ROS manda las rotaciones como cuaterniones en float32, cuyo épsilon es $1,2e-07$, y eso se propaga por las seis juntas. O sea que lo que se está midiendo ahí es la precisión del transporte de ROS, no la del modelo.

Evidencia guardada en `docs/validacion_hito1.txt`.

I. Discrepancia y Debugging

Tuvimos tres. Las dos primeras daban errores enormes y nos costaron bastante.

Discrepancia 1 — dos nodos publicando las juntas

Síntoma: el validador reportaba error posicional máximo de 1,110 m, del orden del alcance del robot. La pose analítica no tenía nada que ver con la que leíamos de TF.

Revisado: [] Sintaxis [] Parent/Child [] Origin xyz/rpy [] Axis [] Visual origin [x] ROS [] DH vs URDF

Qué pasaba: el `joint_state_publisher` seguía corriendo y publicaba $q = 0$ todo el tiempo. Cada configuración que mandaba el validador era pisada antes de que TF se recompusiera, así que terminábamos comparando la analítica de una q aleatoria contra la TF de $q = 0$.

Corrección y evidencia: separamos los dos modos con el argumento `gui` del launch. Con `gui:=true` se levanta la interfaz de sliders para la demostración, y con `gui:=false` no se levanta ningún publicador de juntas, así el tópico queda libre para el validador. Además le agregamos al script una función que cuenta los publicadores de `/joint_states` y avisa si hay más de uno. Después de eso el error bajó al orden de 10^{-10} m.

Discrepancia 2 — desfase temporal al leer TF

Síntoma: aun sin publicadores en conflicto el error seguía en el orden de 1,217 m y cambiaba de una corrida a otra. El error medio era 0,361 m, que es más o menos la distancia entre dos poses aleatorias cualesquiera.

Revisado: [] Sintaxis [] Parent/Child [] Origin xyz/rpy [] Axis [] Visual origin [x] ROS [] DH vs URDF

Qué pasaba: el script publicaba q , esperaba 10 ms y leía TF con `lookup_transform(..., rospy.time.Time())`, que devuelve la última transformada disponible. Como el `robot_state_publisher` todavía no había procesado el mensaje nuevo, lo que leíamos era la pose de la configuración anterior. Estábamos comparando la analítica de $q[k]$ contra la TF de $q[k-1]$.

Corrección y evidencia: sacamos la espera fija. Ahora cada `JointState` se manda con marca de tiempo, y la función de lectura relee TF hasta que el `header.stamp` de la transformada alcanza al del mensaje publicado. Recién ahí acepta el valor. Si se vence el timeout, la muestra se descarta en vez de promediarse. Por eso quedan 499 lecturas perdidas de 1000: no son mediciones malas, son muestras cuya sincronización no pudimos garantizar. El error quedó en $2,459e-10$ m.

Discrepancia 3 — gimbal lock al convertir a rpy

Síntoma: al sacar los ángulos roll-pitch-yaw del origen de J2 nos daba `NaN`.

Revisado: [] Sintaxis [] Parent/Child [x] Origin xyz/rpy [] Axis [] Visual origin [] ROS [] DH vs URDF

Qué pasaba: $R_x(-\pi/2) \cdot R_z(-\pi/2)$ da una matriz con $R[2][0] = 1$, y la fórmula normal calcula roll y yaw dividiendo por $\cos(\text{pitch})$, que en ese punto vale cero.

Corrección y evidencia: la función `matrix_to_rpy()` detecta el caso ($|R[2][0]| > 1 - 1e-10$), fija `yaw = 0` y pasa todo el giro a `roll` con `arctan2` sobre la primera fila. Verificamos el resultado contra la matriz original y contra el URDF escrito a mano: diferencia 0,00e+00.

J. Conclusión Final (grupo)

Consideramos que nuestro URDF es cinemáticamente defendible porque cada `<origin>` lo derivamos algebraicamente de la tabla D-H con la fórmula $Origin_i = B_{i-1} \cdot R_z(off_i)$, en vez de ajustarlo mirando RViz hasta que se viera bien.

Eso es lo que sostiene todo lo demás. Como la conversión es exacta, que la cadena URDF coincida con el producto 0T_6 no es una aproximación que salió bien, es una consecuencia de cómo se construyó. La verificación interna lo confirma en $3,68e-16$ m, que es el ϵ de float64.

Lo respaldamos con tres pruebas que no dependen una de otra. La primera es la pose en $q = 0$, que se predice sumando cotas del manual y valida la transcripción de la tabla sin ejecutar nada. La segunda es la comparación entre el URDF escrito a mano y el generado por script, que descarta errores de transcripción. La tercera es el barrido de 501 configuraciones contra el árbol TF de ROS, validando posición y orientación, con error máximo de $2,459e-10$ m, unas cuatro mil veces por debajo del umbral.

También dejamos documentadas las discrepancias que fuimos encontrando, incluidos dos errores de más de un metro que tuvimos que diagnosticar y corregir. Nos pareció que un resultado correcto sin mostrar cómo se llegó a él no sería defendible.

Comandos de verificación

```
python3 scripts/fk_irbl20.py           # pose en q = 0 y orígenes intermedios
python3 scripts/generate_urdf.py       # genera el URDF y muestra la procedencia
python3 scripts/compare_urdf.py        # compara manual contra generado
check_urdf urdf/irbl20.urdf            # validez XML y árbol

ros2 launch irbl20_description display.launch.py gui:=false
python3 scripts/validate_tf.py 1000    # validación TF contra analítica
```